ENGENHARIA DE SOFTWARE — SISTEMAS DE INFORMAÇÃO

Atividade Prática: Spec-Driven Development (SDD)

Desenvolvimento Orientado por Especificações

IDENTIFICAÇÃO DO TRABALHO & ENTREGA

Aluno: Jonathan Santos (Jonathan G. O. Santos)

Projeto: Sistema de Chamados de Suporte (Help Desk MVP)

Repositório GitHub: https://github.com/JonathanGOSantos/help-desk

1. RESUMO EXECUTIVO E VISÃO DO SISTEMA

Este projeto implementa o Produto Mínimo Viável (MVP) de um Sistema de Chamados de Suporte (Help Desk), substituindo processos informais de abertura de chamados (e-mails dispersos e conversas de corredor) em uma empresa. O desenvolvimento seguiu estritamente o ciclo metodológico do Spec-Driven Development (SDD): Problema -> Especificação -> Esclarecimento -> Planejamento -> Tarefas -> Implementação -> Testes -> Validação.

Arquitetura e Tecnologias Adotadas

- **Back-end:** Node.js com Express para fornecimento de API RESTful enxuta e robusta.
- **Front-end:** HTML5 semântico, CSS3 moderno responsivo e JavaScript ES6+ vanilla com Fetch API nativa.
- **Persistência:** Base estruturada em JSON (src/data/chamados.json) com operações atômicas de leitura e escrita.
- **Testes Automatizados:** Suíte de 28 testes de aceitação cobrindo 100% dos critérios definidos nas especificações.

Funcionalidades Entregues no Repositório

- **01 — Criar Chamado:** Registro com solicitante, categoria homologada, título e descrição. Status inicial sempre "Aberto".
- **02 — Listar Chamados:** Visualização tabular em ordem cronológica decrescente com badges de status coloridos e KPIs.
- **03 — Consultar Chamado:** Modal acessível de detalhes exibindo a descrição integral preservada e metadados de auditoria.
- **04 — Atualizar Atendimento:** Transições de status e registro de parecer técnico da solução, com timestamps automáticos.
- **05 — Filtro e Busca (Evolução):** Pesquisa em tempo real e filtros múltiplos simultâneos por status, cumprindo a Etapa 23 do SDD.

2. RELATO SOBRE O USO DE INTELIGÊNCIA ARTIFICIAL (AI-USAGE)

Em atendimento à Etapa 27 do roteiro, relata-se como ferramentas de IA Generativa foram incorporadas ao processo de engenharia, resguardando o papel do desenvolvedor como autoridade sobre as decisões do projeto.

Ferramenta Utilizada e Etapas de Atuação

Foi utilizado o ambiente Google Antigravity (com o modelo Gemini). A ferramenta atuou como assistente técnico de pair programming nas seguintes fases: revisão inicial de project.md e roadmap.md (Etapa 4); levantamento de ambiguidades e perguntas de esclarecimento (Etapa 6); decomposição de tarefas atômicas (Etapas 7 e 8); sugestão de estruturas de código e rotas REST (Etapa 9); e formulação das asserções da suíte de testes de aceitação (Etapa 10).

Sugestões da IA que Foram Rejeitadas (Controle de Engenharia)

Para assegurar fidelidade aos requisitos de negócio e evitar complexidade desnecessária, diversas propostas da IA foram ativamente rejeitadas:

• **Rejeição de JWT e PostgreSQL no MVP:** A IA sugeriu autenticação com tokens JWT e banco PostgreSQL com Prisma ORM. Foi rejeitado porque project.md definia autenticação e SGBDs pesados explicitamente fora do escopo inicial.

• **Rejeição de Status Arbitrários:** A IA sugeriu um endpoint aberto permitindo criar status livres ("Pausado", "Aguardando Peça"). Rejeitado para garantir o fluxo canônico homologado da empresa (Aberto -> Em atendimento -> Concluído / Cancelado).

• **Rejeição de Alteração da Especificação diante de Falha:** Quando um teste falhou na validação de parecer técnico, a IA propôs relaxar o critério para aprovar o código. A sugestão foi refutada, mantendo o critério e corrigindo o código da validação.

Problemas Identificados nas Respostas da IA

1. Tendência a assumir regras tácitas inexistentes (como inventar prioridades "Alta/Média" quando a especificação ainda não contemplava prioridades);

2. Validações superficiais de strings (uso de verificações simples que permitiam strings preenchidas exclusivamente por espaços em branco, corrigidas pela exigência de trim());

3. Confusão inicial entre testes unitários com mocks vazios e testes de aceitação reais que efetuam requisições HTTP fim a fim.

Verificação do Código e Responsabilidade Decisória

Todo o código gerado foi revisado manualmente e verificado pela suíte automatizada de 28 testes de aceitação em tests/test-acceptance.js. Permaneceram sob estrita responsabilidade humana: a delimitação de escopo do MVP, a garantia de imutabilidade do relato do solicitante, as regras de negócio para transição de status e a escolha de uma arquitetura leve e portável.

3. RESPOSTAS ÀS QUESTÕES PARA REFLEXÃO (ETAPA 28)

Questão 1: Em que momentos a especificação evitou interpretações diferentes sobre o comportamento?

A especificação evitou divergências cruciais em dois pontos centrais:

a) Status inicial do chamado (Feature 01): Garantiu que todo chamado recém-criado nasça categoricamente com status "Aberto", impedindo que formulários da interface ou requisições maliciosas injetassem status prematuros como "Em atendimento" ou "Concluído";

b) Exigência de parecer para conclusão (Feature 04): Definiu que a alteração de status para "Concluído" ou "Cancelado" exige obrigatoriamente o preenchimento de notas técnicas de atendimento com no mínimo 5 caracteres, sanando a dor crônica da empresa de encerramentos sem registro histórico.

Questão 2: Quais ambiguidades foram identificadas antes da implementação?

Na sessão de esclarecimento (Etapa 6), foram sanadas três ambiguidades importantes:

- **Limites de caracteres:** O requisito dizia apenas "título e descrição". Fixou-se 5 a 100 caracteres para título e 10 a 2000 para descrição, evitando cadastros inúteis como "deu pau";
- **Categorias válidas:** Homologou-se um conjunto fechado de 5 categorias (Hardware, Software, Rede e Internet, Acesso e Permissões, Outros), evitando termos livres inconsistentes;
- **Strings com apenas espaços:** Estabeleceu-se o uso de trim() mandatório no cliente e servidor para barrar envios de campos visualmente vazios.

Questão 3: Houve alguma situação em que a IA sugeriu uma solução fora da especificação?

Sim. Ao codificar o endpoint de atualização, a IA propôs um método genérico que permitia ao cliente sobrescrever o texto da descrição original e alterar manualmente os timestamps de auditoria (criadoEm). Essa sugestão foi prontamente descartada porque violava a regra de integridade do relato original do funcionário e o controle estrito de timestamps gerados exclusivamente pelo servidor.

Questão 4: O que aconteceu quando uma implementação não passou em um critério de aceitação?

Durante os testes da Feature 04, o critério CA03 (bloqueio de conclusão sem notas) falhou porque a validação aceitava strings vazias (""). Respeitando a Seção 25 do SDD ("A especificação como fonte de verdade"), não alteramos o critério. A equipe investigou a falha no repositório, ajustou a lógica defensiva para checar o comprimento após trim(), e reexecutou os testes até obter aprovação integral.

Questão 5: Qual foi a relação entre requisitos, critérios de aceitação e testes?

A relação seguiu uma cadeia de rastreabilidade formal e objetiva:

- **Requisito:** Declaração de negócio em nível qualitativo ("O técnico deve documentar a resolução antes de encerrar o ticket");
- **Critério de Aceitação:** Condição mensurável e binária ("Ao submeter status Concluído sem notas, a API deve retornar HTTP 400");
- **Teste de Aceitação:** Tradução executável em código no script tests/test-acceptance.js verificando o status code retornado.

Questão 6: O trabalho de especificação tornou a implementação mais fácil ou mais difícil? Por quê?

Tornou a implementação significativamente mais fácil e veloz. Embora a fase de especificação e esclarecimento exija esforço e disciplina preliminares, a fase de escrita de código tornou-se mecânica e sem retrabalho. Como as rotas, tipos, códigos HTTP (200, 201, 400, 404) e invariantes estavam pré-definidos, o desenvolvimento fluiu sem hesitação e sem necessidade de refatorações de arquitetura.

Questão 7: Quais partes do processo poderiam ser automatizadas com ferramentas de IA?

• **Geração de dados de teste:** Criação de cenários sintéticos e casos de borda (inputs nulos, caracteres de escape, textos longos);

• **Tradução de critérios em código:** Conversão automática de critérios de aceitação formalizados em asserções de testes executáveis (scripts HTTP);

• **Esqueletos repetitivos:** Criação de código boilerplate (rotas REST CRUD, formulários HTML baseados em schemas).

Questão 8: Quais decisões ainda precisam necessariamente de participação humana?

• **Escopo e Visão:** Definição da priorização de escopo e visão do produto (o que entra no MVP e o que fica para o futuro);

• **Regras de Negócio:** Arbitragem de regras de negócio fundamentais da organização;

• **Ética e Governança:** Decisão entre manter a especificação ou alterar o código quando ocorrem falhas;

• **Experiência do Usuário:** Avaliação ergonômica de UX no fluxo real de trabalho dos colaboradores.

Questão 9: O que você faria diferente se esse sistema fosse um produto real utilizado por uma empresa?

1. Autenticação e SSO: Integração com Microsoft Entra ID / Google Workspace para login corporativo;
2. Banco de Dados Relacional Concorrente: Substituição do arquivo JSON por PostgreSQL com migrations e pooling;
3. Mensageria e Notificações: Integração assíncrona (RabbitMQ/Redis) para disparar e-mails e alertas corporativos;
4. Anexos em Nuvem: Upload de fotos de telas e relatórios de erro em object storage (S3/GCS);
5. Gestão de SLAs: Cronômetros regressivos e métricas de tempo médio de atendimento (TMA) e resolução (TMR).

4. RESULTADO DOS TESTES DE ACEITAÇÃO & CONCLUSÃO

SUÍTE DE TESTES (npm test): 28/28 TESTES PASSARAM COM SUCESSO

• Feature 01 (Criar Chamado): 9 critérios validados (CA01 a CA06 + validações de segurança e regras).
• Feature 02 (Listar Chamados): 5 critérios validados (ordenação cronológica decrescente e campos).
• Feature 03 (Consultar Chamado): 5 critérios validados (IDs válidos, inválidos e retorno 404).
• Feature 04 (Atualizar Atendimento): 9 critérios validados (transições de status, timestamps e parecer obrigatório).

Conclusão: A prática demonstrou que a abordagem Spec-Driven Development transforma a especificação em um contrato claro e verificável entre a necessidade do usuário e a implementação técnica, conferindo previsibilidade, rastreabilidade e rigor de engenharia ao desenvolvimento de software.